

DATA & AI STUDY NOTES · NOTEBOOK 05

AI-Augmented Data Quality & Validation

dbt, statistical validation, and a local copilot that knows when to refuse

Author Nikhil Sinha

Repository github.com/Nikhil201716/05-AI-Augmented-Data-Quality-Validation-Framework

Stack dbt · DuckDB · Pandera · ChromaDB · Ollama (qwen2.5:0.5b) · Streamlit

Edition 1.0

How to read this book

This book is written to be read from the beginning by someone who has not seen the project and does not yet know the techniques it uses. It is not API documentation - it is closer to a course text built around one real working system.

Two conventions run throughout. Callouts marked **WHY** explain a design decision and name the alternative that was rejected; callouts marked **WARNING** mark a trap that is easy to fall into and hard to notice afterwards. There is a chapter of pure terminal commands so the project can be reproduced without reading anything else, and a chapter of exercises with worked solutions.

Every code listing is extracted from the repository at the moment this book is built, so nothing here can drift out of step with the code it describes. If a listing looks wrong, the code is wrong.

All data in this project is synthetic. No real customer, transaction, or company data appears anywhere.

Contents

1. The Business Problem

- 1.1 Two different questions about the same data
- 1.2 What was built and measured

2. dbt: Transformation as a Graph of Models

- 2.1 The idea in one sentence
- 2.2 `ref()` builds the graph
- 2.3 The three layers, and what each one is for
- 2.4 Materialisation, and why DuckDB

3. Tests as Executable Contracts

- 3.1 A dbt test is a query that should return nothing
- 3.2 What actually ran
- 3.3 Severity is a design decision, not a default
- 3.4 `dbt run` then `dbt test` - the gap this creates

4. Two Validators, Different Jobs

- 4.1 The question each tool can answer
- 4.2 The five statistical checks
- 4.3 Check 1 - plausible ranges, and the pricing bug it found
- 4.4 Check 3 - the z-score, and its assumption

5. A Guided Tour of the Codebase

- 5.1 How to read this tour
- 5.2 `scripts/generate_data.py`
- 5.3 `scripts/load_raw_to_duckdb.py`
- 5.4 `dbt_project/models/staging/stg_orders.sql`
- 5.5 `dbt_project/models/intermediate/int_orders_enriched.sql`
- 5.6 `dbt_project/models/marts/fct_orders.sql`
- 5.7 `dbt_project/tests/assert_no_extreme_line_totals.sql`
- 5.8 `validation/pandera_checks.py`

- 5.9 rag/build_knowledge_base.py
- 5.10 rag/copilot.py
- 5.11 rag/suggest_validation_rules.py
- 5.12 scripts/run_pipeline.py
- 5.13 dashboard/streamlit_app.py

6. The Incident Two Checks Could Not See

- 6.1 The incident
- 6.2 The arithmetic
- 6.3 Stronger: one of these checks can never fire
- 6.4 Why lowering the threshold is the wrong fix

7. Retrieval-Augmented Generation, From First Principles

- 7.1 The problem RAG solves
- 7.2 Embeddings: meaning as geometry
- 7.3 Chunking, and what goes in the knowledge base
- 7.4 Retrieval

8. The Guardrail: Teaching a Model to Refuse

- 8.1 Retrieval always returns something
- 8.2 Refusal as a distance threshold
- 8.3 What was measured, and what it is worth
- 8.4 Choosing a 0.5B model on purpose

9. AI-Assisted Rule Authoring, Including Where It Fails

- 9.1 The task
- 9.2 Where the small model gets it wrong

10. Worked Examples

- 10.1 Example 1 - The rate that could not fire
- 10.2 Example 2 - Proving the check is incapable
- 10.3 Example 3 - Calibrating the guardrail threshold

11. Running the Project, Step by Step

- 11.1 Prerequisites

- 11.2 Step 1 - Clone and enter the repository
- 11.3 Step 2 - Create an isolated environment
- 11.4 Step 3 - Install dependencies
- 11.5 Step 4 - Run the pipeline, stage by stage
- 11.6 Run everything in one command
- 11.7 Ask the copilot from the command line
- 11.8 Draft validation rules with the local model
- 11.9 Launch the dashboard and chat UI
- 11.10 Verify reproducibility from nothing
- 11.11 Command reference card
- 11.12 Troubleshooting

12. The Dashboard and the Transcripts

- 12.1 One warehouse, two tabs
- 12.2 Why the transcripts matter more than the dashboard

13. Results, and What They Do Not Establish

- 13.1 What was proved
- 13.2 What this does not establish

14. Exercises, with Solutions

- 14.1 Exercises
- 14.2 Solutions

15. Appendix

- 15.1 A - Repository map
- 15.2 B - Glossary
- 15.3 C - Reproducibility
- 15.4 D - Where this leads next

CHAPTER 1

The Business Problem

A failing test tells the engineer who wrote it that something broke. It tells nobody else anything at all.

1.1 Two different questions about the same data

A data platform with tests produces output like `relationships_fct_orders_product_id: Got 21 results, configured to fail if != 0`. To the engineer who wrote that test it is a complete sentence. To the product manager who needs to know whether this morning's revenue number can go in a board deck, it is noise.

So the product manager messages the data engineer, and waits. That queue - one person who can translate test output into business meaning - is the actual bottleneck in most data teams.

This project builds both halves: a properly layered dbt pipeline with real tests, **and** a local AI copilot that anyone can ask in plain English, with every answer grounded in and cited against the actual current test results.

WHY THE TWO HALVES BELONG IN ONE PROJECT

Data engineering and GenAI are usually presented as separate tracks. The combination is more useful than either alone, and it is where the interesting engineering is: a copilot with nothing real to retrieve is a chatbot, and a test suite nobody can interpret is a bottleneck.

The joint version has a property neither has separately - **every answer the copilot gives is checkable against a file the pipeline produced**, because the knowledge base is built from the test output rather than from prose about it.

1.2 What was built and measured

1.1 Measured results

Metric	Value
Orders in the final warehouse	33,542
dbt tests	24 passed / 1 warning / 2 failed, of 27
Statistical checks	3 passed / 2 failed, of 5
Incident isolated to	2026-06-17 - 341 orders vs a 222 average, z=4.56
Copilot: grounded answers and correct refusals	6 / 6 tested
Clean-rebuild reproducibility	identical row counts, tests and z-score

FAILING TESTS ARE THE DELIVERABLE HERE, NOT A DEFECT

Two dbt tests fail and two statistical checks fail. That is the correct outcome: the dataset contains a deliberately injected incident, and a validation framework that reported everything clean would be broken.

What matters is *which* checks caught it and which did not - and the two that passed are the most interesting result in the project.

CHAPTER 2

dbt: Transformation as a Graph of Models

Models as SELECT statements, ref() and the inferred dependency graph, materialisation, and why the staging/intermediate/marts layering is not bureaucracy.

2.1 The idea in one sentence

A dbt model is a `SELECT` statement in a file, and dbt handles turning it into a table or a view, in the right order.

That sounds small. What it buys is that the transformation layer becomes ordinary source-controlled code - reviewable, testable, diffable - instead of a collection of stored procedures and scheduled queries that nobody can safely change.

A staging model - just a SELECT

dbt_project/models/staging/stg_orders.sql

```
-- Deduplicates on order_id (keeping the first occurrence) and casts
-- types. Deliberately does NOT drop rows with a null customer_id or an
-- invalid quantity - those are flagged (is_missing_customer,
-- is_invalid_quantity) and carried forward so they're visible to tests,
-- Pandera, and the AI copilot instead of silently vanishing.

with deduped as (
  select
    *,
    row_number() over (partition by order_id order by order_timestamp) as rn
  from {{ source('raw', 'raw_orders') }}
)

select
  order_id,
  customer_id,
  cast(order_timestamp as timestamp) as order_timestamp,
  product_id,
  quantity,
  unit_price,
  discount_pct,
  channel,
  cast(is_incident_day as boolean) as is_incident_day,
  (customer_id is null) as is_missing_customer,
  (quantity <= 0 or quantity > 100) as is_invalid_quantity
from deduped
where rn = 1
```

2.2 ref() builds the graph

Models refer to each other with `{{ ref('stg_orders') }}` rather than by table name. That indirection does three things at once.

1. **It builds the dependency graph.** dbt parses every `ref()` and derives the DAG, then runs models in topological order. Nothing has to be scheduled by hand.
2. **It makes environments work.** The same code resolves to a dev schema or a production schema depending on the profile, with no conditionals in the SQL.
3. **It gives you lineage for free.** Because the graph is derived from the code, it cannot drift from the code - the failure mode of every hand-maintained lineage diagram.

THIS IS THE SAME ARGUMENT AS THE AIRFLOW DAG IN PROJECT 4

In both cases the dependency structure is *inferred from how data actually flows* rather than declared separately. A separately declared dependency is a second source of truth, and second sources of truth diverge.

The failure mode when they diverge is identical too: something reads a table that has not been rebuilt yet, and produces stale results without any error.

The fact model, referencing upstream models

dbt_project/models/marts/fct_orders.sql

```

-- The final, business-ready order-level fact table. Everything a BI
-- tool or analyst would query lives here, including the data-quality
-- flags carried up from earlier layers - so "show me every order with
-- a problem" is a single WHERE clause, not a re-run of the pipeline.

with orders as (
    select * from {{ ref('int_orders_enriched') }}
),
payments as (
    select * from {{ ref('stg_payments') }}
)

select
    o.order_id,
    o.customer_id,
    o.email,
    o.country,
    o.order_timestamp,
    o.product_id,
    o.product_name,
    o.category,
    o.quantity,
    o.unit_price,
    o.discount_pct,
    o.line_total,
    o.channel,
    o.is_incident_day,
    o.is_missing_customer,
    o.is_invalid_quantity,
    o.is_orphan_product,
    p.payment_id,
    p.amount as payment_amount,
    p.payment_method,
    p.status as payment_status,
    p.paid_at,
    (p.payment_id is not null) as has_payment,
    (p.payment_id is not null and round(p.amount, 2) != round(o.line_total, 2)) as amount_mismatch
from orders o
left join payments p on o.order_id = p.order_id

```

2.3 The three layers, and what each one is for

2.1 The layering convention

Layer	Job	Rule	In this project
Staging	one model per source table: rename, cast, tidy	no joins, no business logic	<code>stg_customers</code> , <code>stg_products</code> , <code>stg_orders</code> , <code>stg_payments</code>
Intermediate	joins and reusable business logic	not consumed directly by users	<code>int_orders_enriched</code>
Marts	what analysts and dashboards query	shaped for consumption, not for storage	<code>fct_orders</code> , <code>dim_customers</code> , <code>dim_products</code>

THE RULE THAT MAKES STAGING WORTH THE EXTRA FILES

Exactly one staging model per source table, and nothing but renaming and casting in it.

That single constraint means every downstream model reads a consistent, predictable interface, and any change to a source system is absorbed in exactly one file.

Skip the layer and every mart contains its own copy of the source's quirks - the same date-parsing hack in four models, three of which get updated when the source changes. The staging layer is the anti-corruption boundary between systems you do not control and code you do.

The marts follow dimensional modelling: `fct_orders` is a fact table at the order-line grain, `dim_customers` and `dim_products` are dimensions. The same star schema as Projects 3 and 4, arrived at independently, because the shape is genuinely the right one for analytical querying.

2.4 Materialisation, and why DuckDB

Each model is a **view** (recomputed on every query) or a **table** (computed once at build time). Views are always fresh and cost nothing to store; tables cost storage and staleness but are fast to query. Staging models are typically views; marts are typically tables.

DUCKDB IS A REAL SUBSTITUTION, AND A DEFENSIBLE ONE

DuckDB is an in-process analytical database - columnar, vectorised, and requiring no server. It is to Postgres roughly what SQLite is to MySQL, except built for analytics.

It is used here because the series' constraint is that anyone must be able to run these projects for free, forever, on modest hardware. A project requiring a Snowflake account is a project nobody can run. The dbt models are ordinary SQL and the adapter is one line of `profiles.yml` .

CHAPTER 3

Tests as Executable Contracts

Generic tests, singular tests, severity as a design decision, and the gap between dbt run and dbt build.

3.1 A dbt test is a query that should return nothing

The whole model is that simple. A test is SQL selecting the rows that *violate* a rule. Zero rows returned means the rule holds.

A singular test - SQL selecting violations

dbt_project/tests/assert_no_extreme_line_totals.sql

```
-- Singular test: fails if it returns any rows. Catches a pricing-bug
-- style anomaly (a unit-price multiplier error) that no schema test
-- (not_null/unique/relationships) is shaped to catch, because every
-- value here is technically present and technically numeric - it's
-- just wildly outside a plausible business range for this catalog
-- (products range from ~$15 to ~$290).

select *
from {{ ref('int_orders_enriched') }}
where line_total < 0.50 or line_total > 5000
```

Two kinds. **Generic tests** are declared in YAML against a column - `not_null`, `unique`, `accepted_values`, `relationships` - and cover most of what you need. **Singular tests** are hand-written SQL for rules that are specific to your business.

Generic tests declared in YAML

dbt_project/models/staging/_staging.yml

```

version: 2

models:
  - name: stg_customers
    description: One row per customer, lightly cleaned from raw_customers.
    columns:
      - name: customer_id
        tests: [unique, not_null]
      - name: email
        tests: [not_null]

  - name: stg_products
    description: One row per product, lightly cleaned from raw_products.
    columns:
      - name: product_id
        tests: [unique, not_null]

  - name: stg_orders
    description: >
      One row per order (deduplicated on order_id). Rows with a missing
      customer or invalid quantity are KEPT and flagged, not dropped.
    columns:
      - name: order_id
        tests: [unique, not_null]
      - name: product_id
        tests: [not_null]
      - name: quantity
        tests: [not_null]
      - name: order_timestamp
        tests: [not_null]
      - name: customer_id
        tests:
          - not_null:
              config:
                severity: warn
                error_if: ">100"
                warn_if: ">0"
      - name: channel
        tests:
          - accepted_values:

```

DECLARING TESTS BESIDE THE MODEL IS WHAT MAKES THEM CONTRACTS

The YAML sits next to the model it describes. Changing the model means seeing its tests in the same diff, and adding a column with no test is visibly a decision rather than an oversight.

A test suite in a separate repository, run by a separate team, is not a contract - it is a second system that gradually stops matching the first.

3.2 What actually ran

The real dbt test output

```
Done. PASS=24 WARN=1 ERROR=2 SKIP=0 NO-OP=0 REUSED=0 TOTAL=27
```

```
[ERROR]    assert_no_extreme_line_totals: Got 52 results, configured to fail if != 0
[ERROR]    relationships_fct_orders_product_id: Got 21 results, configured to fail if != 0
[WARNING]  not_null_stg_orders_customer_id: Got 22 results, configured to warn if >0
```

Three findings, and the severity of each was chosen deliberately.

3.3 Severity is a design decision, not a default

3.1 Why each finding has the severity it has

Finding	Severity	Reasoning
22 null <code>customer_id</code>	warn	guest checkout is a legitimate part of this business
21 orphaned <code>product_id</code>	error	there is no valid reason an order references a nonexistent product
52 extreme line totals	error	outside any plausible range - a pricing or quantity bug

THE SAME 22 ROWS ARE A WARNING HERE AND PART OF A HARD FAILURE ELSEWHERE

Those 22 null customer IDs are all on the incident day. As a dbt test they are a warning, because nulls in that column are normal at a low background rate. As a *daily rate* they are 6.45% of that day's orders - a serious anomaly.

Neither judgement is wrong. They are answering different questions, and this is the clearest illustration in the project of why one tool is not enough. A rule that is right at the row level can be blind at the aggregate level, and vice versa.

THE COST OF GETTING SEVERITY WRONG IS THE MUTED ALERT

Make guest checkout a hard error and the pipeline fails most days for a normal business event. Within a month the team runs with that test disabled, and the disabling is permanent because nobody remembers why. This is the same alert-fatigue argument as Project 4's two-tier thresholds, arrived at from a different direction.

3.4 dbt run then dbt test - the gap this creates

This project runs `dbt run` and then, separately, `dbt test`. The combined command `dbt build` interleaves them: it builds a model, tests it, and **refuses to build anything downstream if a test fails**.

THE SAME FLAW AS PROJECT 4, IN A DIFFERENT TOOL

Because the tests run after all the models are built, the 21 orphaned rows and the 52 extreme line totals are in `fct_orders` right now. The tests correctly report them and the marts contain them anyway.

Project 4 hit this exact structure: the quality gate ran after the load committed, and rejected rows reached the warehouse. Two different stacks, two different tools, the same architectural mistake - which suggests it is the default shape rather than an oversight, and worth recognising by name.

The reason it is done this way here is stated in the project's own report: `dbt build` would correctly prevent `fct_orders`, `dim_customers` and `dim_products` from being created at all on a run with a hard failure - which is the right production behaviour and would leave a demo with no marts to explore.

THE DISCLOSED TRADE-OFF IS THE POINT

The choice is defensible for a demo and indefensible for production, and the project says so in both the architecture doc and the assessment report, along with the fix - a staged load with an atomic swap, which is write-audit-publish again.

An undisclosed version of this choice would be a project that quietly ships broken data while displaying a testing framework. The disclosure is what makes it a teaching example instead.

CHAPTER 4

Two Validators, Different Jobs

Why dbt tests and a statistical validator are not redundant, what each one can and cannot express, and the five checks in this layer.

4.1 The question each tool can answer

It looks like duplication to run two validation tools. It is not, and the distinction is worth stating precisely because it decides which failures you are capable of noticing.

4.1 The division of labour between the two validators

	dbt tests	Statistical validation (Pandera)
Question	is this structurally valid?	does this look right?
Scope	a row, or a column	a distribution, or a time series
Nature	deterministic - true or false	comparative - relative to a baseline
Example	is <code>customer_id</code> ever null?	is today's volume normal for a Wednesday?
Fails when	a constraint is violated	the shape of the data changes
Blind to	a plausible-but-wrong value	a single malformed row

NEITHER TOOL CAN DO THE OTHER'S JOB, EVEN IN PRINCIPLE

A price of \$4,999 for a \$49.99 item breaks no structural rule. It is not null, it is unique, it is a positive number, it resolves to a real product. Every dbt test passes. Only a check that knows what prices normally look like can flag it.

Conversely, a single row with a null foreign key is invisible to a distributional check - one row does not move a distribution. It is caught precisely by the row-level constraint.

The failures they miss are disjoint, which is the entire argument for the cost of running both.

4.2 The five statistical checks

4.2 The statistical validation layer: 3 passed, 2 failed

#	Check	What it asks	Result
1	plausible_value_ranges	are quantities and prices inside sane bounds?	FAIL - 55 values
2	email_format_valid	do emails match a pattern?	PASS
3	daily_volume_anomaly_detection	is any day's volume statistically abnormal?	FAIL - 2026-06-17, $z=4.56$
4	missing_customer_rate_threshold	is the missing-customer rate under 2%?	PASS - 0.07%
5	orphan_product_rate_threshold	is the orphaned-product rate under 1%?	PASS - 0.06%

CHECKS 4 AND 5 PASSED ON DATA THAT WAS DEMONSTRABLY BROKEN

The same incident that check 3 caught at $z = 4.56$ was sitting in the data while checks 4 and 5 reported it comfortably within tolerance.

This is not a bug in checks 4 and 5. They computed exactly what they were asked to compute, correctly. The next chapter proves that one of them could *never* have fired on this incident, no matter how bad the day was - and that the reason is arithmetic, not tuning.

4.3 Check 1 - plausible ranges, and the pricing bug it found

Two bounds: order quantity in [1, 20], unit price in [1.00, 500.00]. Twenty-five quantity violations and thirty price violations, 55 in total.

Neither bound is a law of nature. Somebody could legitimately order 50 units. The bounds encode a claim about *this business*: an outdoor-gear retailer's orders are small, and its catalogue tops out well below \$500.

A PLAUSIBILITY BOUND MUST BE JUSTIFIED, OR IT IS NOISE

The failure mode is a bound picked by eye that fires constantly on legitimate data, gets widened until it stops firing, and then catches nothing. A bound should come from the observed distribution - a high percentile with headroom - and should be revisited when the business changes.

The honest statement here is that these bounds were chosen to match the generator's parameters, so their calibration is not evidence of anything. On real data they would need to be measured.

4.4 Check 3 - the z-score, and its assumption

For each day, compare its order count to the mean and standard deviation across all days:

$$z_d = \frac{n_d - \bar{n}}{s_n}, \quad \text{flag if } |z_d| > 2.5$$

With $\bar{n} = 222$ and $s_n = 26$, the incident day's 341 orders give

$$z = \frac{341 - 222}{26} = \frac{119}{26} = 4.58$$

- reported as 4.56 by the check, which uses unrounded inputs. Under a normal approximation $|z| > 2.5$ is roughly a 1.2% two-sided event, so on 151 days you would expect about two false flags by chance. Exactly one day fired, and it was the injected one.

THE MEAN AND STANDARD DEVIATION INCLUDE THE INCIDENT

This is a real methodological weakness. The baseline $\bar{n}, s_n$ is computed over *all* days, including 2026-06-17. The outlier inflates s_n , which shrinks its own z-score - the check is biased against detecting exactly what it is looking for.

It fired anyway because the incident is large. A subtler one might not. The standard defences are robust statistics - median and median absolute deviation, which an outlier barely moves - or leave-one-out, computing each day's baseline from the other 150 days. Both are a few lines, and neither is implemented here. It is the first exercise in Chapter 14.

CHAPTER 5

A Guided Tour of the Codebase

*Every module that matters, in the order the pipeline runs them, with its purpose and public surface.
This chapter is generated from the repository, so it cannot describe a function that no longer exists.*

5.1 How to read this tour

The pipeline is linear and the two AI components sit at the end, consuming what the data layers produce. Read in this order and each stage's input is the previous stage's output.

5.2 `scripts/generate_data.py`

Why it exists. The seeded generator, including the deliberately injected single-day incident that the entire validation layer exists to catch.

What the module says about itself. Generates a fresh SYNTHETIC multi-table e-commerce dataset for "NorthPeak Outdoor Gear" (a fictional outdoor/camping equipment retailer) - the raw layer this project's dbt models transform and its validation suite checks. 4 raw source tables: customers, products, orders, payments.

Opening of the module

scripts/generate_data.py

```

"""
generate_data.py
-----
Generates a fresh SYNTHETIC multi-table e-commerce dataset for "NorthPeak
Outdoor Gear" (a fictional outdoor/camping equipment retailer) - the raw
layer this project's dbt models transform and its validation suite
checks. 4 raw source tables: customers, products, orders, payments.

Deliberately includes one clear "incident day" where a simulated batch
job bug corrupts a chunk of that day's orders (nulls, duplicates,
orphaned product references, a pricing bug producing extreme order
values) - this is what gives the dbt tests, the Pandera validation
suite, and later the AI Data Quality Copilot something real and
specific to catch and explain, not just clean data with no story.

No real customer, product, or company data is used anywhere.

Output: data/raw_customers.csv, raw_products.csv, raw_orders.csv, raw_payments.csv
"""

import numpy as np
import pandas as pd
from pathlib import Path
from datetime import datetime, timedelta

SEED = 42
rng = np.random.default_rng(SEED)

DATA_DIR = Path(__file__).resolve().parent.parent / "data"
DATA_DIR.mkdir(exist_ok=True)

START_DATE = datetime(2026, 3, 1)
END_DATE = datetime(2026, 7, 29)
DAYS = pd.date_range(START_DATE, END_DATE, freq="D")
INCIDENT_DAY = datetime(2026, 6, 17) # the simulated batch-job bug date

# -----
# 1. Customers
# -----
N_CUSTOMERS = 4000
COUNTRIES = ["USA", "Canada", "UK", "Germany", "Australia", "France"]
FIRST = ["Alex", "Jordan", "Taylor", "Sam", "Casey", "Morgan", "Riley", "Jamie",
         "Avery", "Quinn", "Drew", "Skyler", "Reese", "Rowan", "Emerson"]
LAST = ["Turner", "Brooks", "Rivera", "Bennett", "Coleman", "Foster", "Hayes",
        "Reed", "Stone", "Ward", "Nash", "Blake", "Cross", "Lane", "Pierce"]

customer_ids = [f"CUST{20000+i}" for i in range(N_CUSTOMERS)]
customers = pd.DataFrame({
    "customer_id": customer_ids,
    "email": [f"{rng.choice(FIRST).lower()}.{rng.choice(LAST).lower()}"
              f"{rng.integers(1,999)}@example.com"
              for _ in range(N_CUSTOMERS)],
    "full_name": [f"{rng.choice(FIRST)} {rng.choice(LAST)}" for _ in range(N_CUSTOMERS)],
    "signup_date": [(START_DATE - timedelta(days=int(d))).date().isoformat()
                    for d in rng.integers(0, 900, size=N_CUSTOMERS)],
    "country": rng.choice(COUNTRIES, size=N_CUSTOMERS, p=[0.42, 0.14, 0.16, 0.12, 0.08, 0.08]),
    "marketing_opt_in": rng.choice([1, 0], size=N_CUSTOMERS, p=[0.62, 0.38]),

```

```

}))
customers.to_csv(DATA_DIR / "raw_customers.csv", index=False)
print(f"Customers: {len(customers)} -> raw_customers.csv")

# -----
# 2. Products
# -----
PRODUCTS = [
    ("Trailhead 2P Tent", "Shelter", 189.99), ("Summit 4P Tent", "Shelter", 289.99),
    ("UltraLight Bivy", "Shelter", 79.99), ("CloudNine Sleeping Bag", "Sleep", 129.99),
    ("Alpine -10C Sleeping Bag", "Sleep", 219.99), ("Foam Sleep Pad", "Sleep", 39.99),
    ("Inflatable Sleep Pad", "Sleep", 89.99), ("60L Trail Backpack", "Packs", 219.99),
    ("35L Daypack", "Packs", 99.99), ("Hydration Vest", "Packs", 74.99),
    ("Titanium Cookset", "Cooking", 64.99), ("Camp Stove", "Cooking", 54.99),
    ("Water Filter Straw", "Cooking", 24.99), ("Collapsible Cookware Set", "Cooking", 49.99),
    ("Trekking Poles (Pair)", "Gear", 59.99), ("Headlamp 400lm", "Gear", 34.99),

```

5.3 scripts/load_raw_to_duckdb.py

Why it exists. Loads the raw CSVs into DuckDB as source tables, the boundary where dbt takes over.

What the module says about itself. The "EL" half of ELT: loads the 4 raw CSVs into DuckDB as raw_* tables, untouched. dbt then owns the "T" half - all cleaning and transformation happens inside dbt models, not here. This mirrors how real data platforms are actually built: an extraction/loading tool (Fivetran, Airbyte, or a custom script) lands raw data in the warehouse, and dbt is only ever responsible for transforming what's already there.

Opening of the module

scripts/load_raw_to_duckdb.py

```

"""
Load_raw_to_duckdb.py
-----
The "EL" half of ELT: Loads the 4 raw CSVs into DuckDB as raw_* tables,
untouched. dbt then owns the "T" half - all cleaning and transformation
happens inside dbt models, not here. This mirrors how real data
platforms are actually built: an extraction/loading tool (Fivetran,
Airbyte, or a custom script) lands raw data in the warehouse, and dbt
is only ever responsible for transforming what's already there.
"""

import duckdb
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
DATA_DIR = ROOT / "data"
DB_PATH = ROOT / "database" / "warehouse.duckdb"
DB_PATH.parent.mkdir(exist_ok=True)

conn = duckdb.connect(str(DB_PATH))

tables = ["raw_customers", "raw_products", "raw_orders", "raw_payments"]
for t in tables:
    csv_path = DATA_DIR / f"{t}.csv"
    conn.execute(f"CREATE OR REPLACE TABLE {t} AS SELECT * FROM
read_csv_auto('{csv_path.as_posix()}')")
    count = conn.execute(f"SELECT COUNT(*) FROM {t}").fetchone()[0]
    print(f"Loaded {t}: {count:,} rows")

conn.close()
print(f"\nRaw tables loaded into {DB_PATH}")

```

5.4 dbt_project/models/staging/stg_orders.sql

Why it exists. A staging model: rename, cast, tidy. No joins, no business logic.

Opening of the module

dbt_project/models/staging/stg_orders.sql

```

-- Deduplicates on order_id (keeping the first occurrence) and casts
-- types. Deliberately does NOT drop rows with a null customer_id or an
-- invalid quantity - those are flagged (is_missing_customer,
-- is_invalid_quantity) and carried forward so they're visible to tests,
-- Pandera, and the AI copilot instead of silently vanishing.

with deduped as (
  select
    *,
    row_number() over (partition by order_id order by order_timestamp) as rn
  from {{ source('raw', 'raw_orders') }}
)

select
  order_id,
  customer_id,
  cast(order_timestamp as timestamp) as order_timestamp,
  product_id,
  quantity,
  unit_price,
  discount_pct,
  channel,
  cast(is_incident_day as boolean) as is_incident_day,
  (customer_id is null) as is_missing_customer,
  (quantity <= 0 or quantity > 100) as is_invalid_quantity
from deduped
where rn = 1

```

5.5 dbt_project/models/intermediate/int_orders_enriched.sql

Why it exists. Where the joins happen and the integrity flags are computed.

Opening of the module

dbt_project/models/intermediate/int_orders_enriched.sql

```

-- Joins orders to customers and products. Uses LEFT JOINs deliberately
-- (not INNER) so that orders referencing a non-existent product
-- (is_orphan_product) or a missing customer stay visible in the output
-- with a flag, instead of silently disappearing from every downstream
-- report the moment a join fails to match.

with orders as (
  select * from {{ ref('stg_orders') }}
),
customers as (
  select * from {{ ref('stg_customers') }}
),
products as (
  select * from {{ ref('stg_products') }}
)

select
  o.order_id,
  o.customer_id,
  c.email,
  c.country,
  o.order_timestamp,
  o.product_id,
  p.product_name,
  p.category,
  o.quantity,
  o.unit_price,
  o.discount_pct,
  round(o.quantity * o.unit_price * (1 - o.discount_pct), 2) as line_total,
  o.channel,
  o.is_incident_day,
  o.is_missing_customer,
  o.is_invalid_quantity,
  (o.product_id is not null and p.product_id is null) as is_orphan_product
from orders o
left join customers c on o.customer_id = c.customer_id
left join products p on o.product_id = p.product_id

```

5.6 dbt_project/models/marts/fct_orders.sql

Why it exists. The fact table analysts and the dashboard query.

Opening of the module

dbt_project/models/marts/fct_orders.sql

```

-- The final, business-ready order-level fact table. Everything a BI
-- tool or analyst would query lives here, including the data-quality
-- flags carried up from earlier layers - so "show me every order with
-- a problem" is a single WHERE clause, not a re-run of the pipeline.

with orders as (
    select * from {{ ref('int_orders_enriched') }}
),
payments as (
    select * from {{ ref('stg_payments') }}
)

select
    o.order_id,
    o.customer_id,
    o.email,
    o.country,
    o.order_timestamp,
    o.product_id,
    o.product_name,
    o.category,
    o.quantity,
    o.unit_price,
    o.discount_pct,
    o.line_total,
    o.channel,
    o.is_incident_day,
    o.is_missing_customer,
    o.is_invalid_quantity,
    o.is_orphan_product,
    p.payment_id,
    p.amount as payment_amount,
    p.payment_method,
    p.status as payment_status,
    p.paid_at,
    (p.payment_id is not null) as has_payment,
    (p.payment_id is not null and round(p.amount, 2) != round(o.line_total, 2)) as amount_mismatch
from orders o
left join payments p on o.order_id = p.order_id

```

5.7 dbt_project/tests/assert_no_extreme_line_totals.sql

Why it exists. A singular test - hand-written SQL selecting the rows that violate a business-specific rule.

Opening of the module

dbt_project/tests/assert_no_extreme_line_totals.sql

```

-- Singular test: fails if it returns any rows. Catches a pricing-bug
-- style anomaly (a unit-price multiplier error) that no schema test
-- (not_null/unique/relationships) is shaped to catch, because every
-- value here is technically present and technically numeric - it's
-- just wildly outside a plausible business range for this catalog
-- (products range from ~$15 to ~$290).

select *
from {{ ref('int_orders_enriched') }}
where line_total < 0.50 or line_total > 5000

```

5.8 validation/pandera_checks.py

Why it exists. The statistical layer: plausible ranges, the daily z-score, and the two aggregate rate checks whose blindness is the subject of Chapter 6.

What the module says about itself. Statistical and business-rule validation, deliberately covering DIFFERENT ground than dbt's schema tests rather than duplicating them:

Public functions in validation/pandera_checks.py

Function	What it does
record(check_name, layer, passed, detail)	—

Opening of the module

validation/pandera_checks.py

```

"""
pandera_checks.py
-----

Statistical and business-rule validation, deliberately covering DIFFERENT
ground than dbt's schema tests rather than duplicating them:

- dbt tests answer: "is this column ever null/duplicated?", "does this
  foreign key always resolve?" - structural, row-level, deterministic.
- Pandera here answers: "is this value plausible?", "does today's
  volume/behavior look statistically normal compared to recent
  history?" - distributional, comparative, judgment-based checks that
  a simple not_null/unique test can't express.

This is exactly the kind of two-tool setup a real data platform runs:
dbt owns structural correctness, a statistical validator owns "does this
look right" - and this project intentionally keeps both, because a real
pricing bug (Section: extreme_unit_price) or a real traffic anomaly
(Section: daily volume) can pass every structural test while still being
very wrong.

NOTE ON TOOLING: this project originally targeted Great Expectations for
this layer (matching common industry usage), but GX's latest available
release in this environment pins numpy<2.0, which cannot build from
source on Python 3.14 here (no working C compiler). Pandera is a modern,
actively maintained, increasingly widely used alternative for exactly
this use case - a real, defensible substitution, not a downgrade in
rigor. See README Section 9 for the full note.

Output:
- reports/pandera_validation_results.json (structured, for the RAG
  knowledge base and the dashboard)
- reports/pandera_validation_summary.md (human-readable)
"""

import json
from pathlib import Path

import duckdb
import numpy as np
import pandas as pd
import pandera.pandas as pa
from pandera.pandas import Check, Column, DataFrameSchema

ROOT = Path(__file__).resolve().parent.parent
DB_PATH = ROOT / "database" / "warehouse.duckdb"
REPORTS_DIR = ROOT / "reports"
REPORTS_DIR.mkdir(exist_ok=True)

conn = duckdb.connect(str(DB_PATH), read_only=True)
orders = conn.execute("SELECT * FROM main.fct_orders").fetchdf()
customers = conn.execute("SELECT * FROM main.stg_customers").fetchdf()
conn.close()

results = []

def record(check_name, layer, passed, detail, n_failing=0):

```

```

results.append({
    "check_name": check_name,
    "layer": layer,
    "status": "PASS" if passed else "FAIL",
    "detail": detail,
    "n_failing_rows": int(n_failing),
})
icon = "PASS" if passed else "FAIL"
print(f"[{icon}] {check_name}: {detail}")

# =====
# 1. Schema-level statistical checks on fct_orders (row-level, but checking
#    PLAUSIBILITY, not just presence - complementary to dbt's not_null/unique)
# =====

```

5.9 rag/build_knowledge_base.py

Why it exists. Embeds the pipeline's own output into ChromaDB, so the copilot cannot be staler than the last run.

What the module says about itself. Assembles the knowledge base the AI Data Quality Copilot retrieves from, and embeds it into a persistent ChromaDB collection using Chroma's built-in ONNX MiniLM embedding function (no torch/sentence-transformers needed - a deliberate choice on RAM-constrained hardware, see README).

Public functions in rag/build_knowledge_base.py

Function	What it does
add_doc(doc_id, text)	—

Opening of the module

rag/build_knowledge_base.py

```

"""
build_knowledge_base.py
-----
Assembles the knowledge base the AI Data Quality Copilot retrieves from,
and embeds it into a persistent ChromaDB collection using Chroma's
built-in ONNX MiniLM embedding function (no torch/sentence-transformers
needed - a deliberate choice on RAM-constrained hardware, see README).

Four kinds of documents go in, each written to answer a different kind
of real question a teammate would actually ask:

1. dbt test results      -> "why did X fail / did X pass?"
2. Pandera check results -> "does anything look statistically off?"
3. Model documentation  -> "what does model Y even contain?"
4. A root-cause runbook  -> "what usually causes an orphaned product?"
5. A computed incident summary, built from REAL query results against
   the warehouse (not hand-written), so the copilot's most detailed
   answer is grounded in the actual data, not a canned story.

Run AFTER: scripts/generate_data.py, scripts/load_raw_to_duckdb.py,
dbt run, dbt test, validation/pandera_checks.py.
"""

import json
from pathlib import Path

import chromadb
import duckdb
import pandas as pd

ROOT = Path(__file__).resolve().parent.parent
DBT_TARGET = ROOT / "dbt_project" / "target"
REPORTS_DIR = ROOT / "reports"
DB_PATH = ROOT / "database" / "warehouse.duckdb"
CHROMA_PATH = ROOT / "rag" / "knowledge_base" / "chroma_store"
CHROMA_PATH.parent.mkdir(parents=True, exist_ok=True)

documents = [] # List of (id, text, metadata)

def add_doc(doc_id, text, **metadata):
    documents.append((doc_id, text.strip(), metadata))

# =====
# 1. dbt test results
# =====
DBT_TEST_EXPLANATIONS = {
    "assert_no_extreme_line_totals": (
        "Checks that every order's line_total falls between $0.50 and $5000, a plausible "
        "range for the NorthPeak Outdoor Gear catalog (products range roughly $15-$290). "
        "A failure here almost always indicates a PRICING BUG - most commonly a unit-price "
        "multiplier applied incorrectly upstream (e.g. a price stored in cents being read as "
        "dollars, or vice versa) - not a normal data-entry typo."
    ),
    "assert_valid_order_dates": (
        "Checks that no order is dated before 2025-01-01 or in the future. A failure would "

```

```

        "suggest a timezone bug or a corrupted timestamp field upstream."
    ),
    "not_null_stg_orders_customer_id": (
        "Checks that every order has a customer_id. This is set to WARN severity rather than "
        "hard-fail, because a small background rate of missing customer references can happen "
        "from anonymous/guest checkouts. A sudden SPIKE in this warning's count on one specific "
        "day (rather than a steady background rate) points to a CRM/session-linking bug in the "
        "order-capture system on that day, not guest checkouts."
    ),
    "relationships_fct_orders_product_id__product_id__ref_stg_products_": (
        "Checks that every order's product_id actually exists in the product catalog "
        "(stg_products). A failure means orders reference a PRODUCT THAT DOESN'T EXIST - "
        "typically caused by a catalog sync issue: either the product was deleted/retired "
        "after orders referenced it, or the order-capture system wrote a product_id before "
        "the catalog table finished syncing that day."
    )
)

```

5.10 rag/copilot.py

Why it exists. Retrieval, the distance guardrail, and grounded generation with a local Ollama model.

What the module says about itself. The retrieval + generation core of the AI Data Quality Copilot.

Public functions in rag/copilot.py

Function	What it does
retrieve(question, top_k)	—
ask(question, model)	—

Opening of the module

rag/copilot.py

```

"""
copilot.py
-----
The retrieval + generation core of the AI Data Quality Copilot.

Pipeline: question -> embed (Chroma's built-in ONNX MiniLM) -> retrieve
top-k chunks -> hallucination guardrail (refuse if nothing relevant
enough was found) -> build a grounded prompt -> generate an answer with
a local Ollama model -> return the answer WITH its sources, so a user
can verify the claim instead of just trusting it.

Model: qwen2.5:0.5b via Ollama, run entirely locally. Chosen specifically
for its small memory footprint (~400MB) - this project was built and
tested on a 6GB-RAM machine, so the model had to be small enough to
actually load and run there. See README Section 9 for the full note on
this tradeoff (a bigger model would answer more fluently, but the whole
point of a local-first copilot is that it has to run on modest hardware
someone actually owns, not just a well-resourced dev machine).
"""

from pathlib import Path

import chromadb
import ollama

ROOT = Path(__file__).resolve().parent.parent
CHROMA_PATH = ROOT / "rag" / "knowledge_base" / "chroma_store"

OLLAMA_MODEL = "qwen2.5:0.5b"
TOP_K = 4
DISTANCE_GUARDRAIL_THRESHOLD = 1.4 # calibrated empirically - see docs/architecture.md

SYSTEM_PROMPT = (
    "You are a Data Quality Copilot for the NorthPeak Outdoor Gear data warehouse. "
    "Answer the user's question using ONLY the CONTEXT provided below - do not use "
    "any outside knowledge, and do not make up numbers or test names that are not in "
    "the context. Be concise (2-4 sentences). If the context includes specific counts "
    "or percentages, cite them. If asked something the context doesn't cover, say so "
    "plainly instead of guessing."
)

_client = None
_collection = None

def _get_collection():
    global _client, _collection
    if _collection is None:
        _client = chromadb.PersistentClient(path=str(CHROMA_PATH))
        _collection = _client.get_collection("data_quality_knowledge_base")
    return _collection

def retrieve(question: str, top_k: int = TOP_K):
    collection = _get_collection()
    res = collection.query(query_texts=[question], n_results=top_k)
    hits = []

```

```

for doc, dist, meta, doc_id in zip(
    res["documents"][0], res["distances"][0], res["metadatas"][0], res["ids"][0]
):
    hits.append({"id": doc_id, "text": doc, "distance": dist, "metadata": meta})
return hits

def ask(question: str, model: str = OLLAMA_MODEL) -> dict:
    hits = retrieve(question)

    if not hits or hits[0]["distance"] > DISTANCE_GUARDRAIL_THRESHOLD:
        return {
            "answer": "I don't have information on that in the current data quality knowledge base. "
            "Try asking about a specific dbt test, a Pandera check, a model, or the known "
            "2026-06-17 incident.",

```

5.11 rag/suggest_validation_rules.py

Why it exists. The second GenAI use case: drafting candidate validation rules for human review.

What the module says about itself. AI-assisted validation-rule authoring: given a table's column names, types, and a small sample of real values, asks the local LLM to DRAFT candidate Pandera checks - accelerating the boring part of writing a validation suite (staring at a new table guessing plausible ranges) without ever auto-applying anything.

Public functions in rag/suggest_validation_rules.py

Function	What it does
suggest_rules_for_table(table_name, model)	—

Opening of the module

rag/suggest_validation_rules.py

```

"""
suggest_validation_rules.py
-----
AI-assisted validation-rule authoring: given a table's column names,
types, and a small sample of real values, asks the local LLM to DRAFT
candidate Pandera checks - accelerating the boring part of writing a
validation suite (staring at a new table guessing plausible ranges)
without ever auto-applying anything.

This is deliberately human-in-the-loop: the output is written to a
review file, not wired into the validation suite. A data engineer reads
it, keeps what's sensible, discards what isn't, and only then adds it
to validation/pandera_checks.py by hand. This mirrors how AI-assisted
tooling is actually used responsibly on real data teams in 2025/2026 -
as a drafting aid for a human who stays the final reviewer, not as an
unsupervised decision-maker over what gets treated as "valid" data.

Usage:
    python rag/suggest_validation_rules.py raw_orders
"""

import sys
from pathlib import Path

import duckdb
import ollama

ROOT = Path(__file__).resolve().parent.parent
DB_PATH = ROOT / "database" / "warehouse.duckdb"
OUT_DIR = ROOT / "reports" / "ai_suggested_rules"
OUT_DIR.mkdir(parents=True, exist_ok=True)

OLLAMA_MODEL = "qwen2.5:0.5b"

PROMPT_TEMPLATE = """Table name: {table_name}
Columns: {columns}

Sample rows:
{sample_rows}

Task: write one validation rule for EACH column listed above. For number columns, give a min and
max value based on what you see in the sample rows. For text columns, list the exact values you
see. Do not invent column names that are not listed above.
Format: "<column_name>: <rule>"
"""

def suggest_rules_for_table(table_name: str, model: str = OLLAMA_MODEL) -> str:
    conn = duckdb.connect(str(DB_PATH), read_only=True)
    schema_df = conn.execute(f"DESCRIBE {table_name}").fetchdf()
    sample_df = conn.execute(f"SELECT * FROM {table_name} USING SAMPLE 5 ROWS").fetchdf()
    conn.close()

    columns = ", ".join(f"{r.column_name}: {r.column_type}" for r in schema_df.itertuples())
    sample_rows = sample_df.to_string(index=False)

    prompt = PROMPT_TEMPLATE.format(table_name=table_name, columns=columns, sample_rows=sample_rows)

```

```

response = ollama.chat(
    model=model,
    messages=[{"role": "user", "content": prompt}],
    options={"temperature": 0.2, "num_predict": 300},
)
suggestions = response["message"]["content"].strip()

out_path = OUT_DIR / f"{table_name}_suggested_rules.md"
with open(out_path, "w", encoding="utf-8") as f:
    f.write(f"# AI-Suggested Validation Rules: {table_name}\n\n")
    f.write("**Status: DRAFT - requires human review before adoption.** These suggestions were "
            "generated by a local LLM (qwen2.5:0.5b) from the table's schema and a small sample "
            "of real rows. They are a starting point for a data engineer to review, not a "
            "validated or automatically-applied ruleset.\n\n")

```

5.12 scripts/run_pipeline.py

Why it exists. Runs every stage in order - generation, load, dbt, validation, knowledge base.

What the module says about itself. One-command orchestrator: regenerates the entire project from a single source of truth - synthetic raw data -> DuckDB -> dbt (run + test) -> Pandera statistical validation -> RAG knowledge base rebuild.

Public functions in scripts/run_pipeline.py

Function	What it does
run(label, cmd, cwd, env)	—

Opening of the module

scripts/run_pipeline.py

```

"""
run_pipeline.py
-----
One-command orchestrator: regenerates the entire project from a single
source of truth - synthetic raw data -> DuckDB -> dbt (run + test) ->
Pandera statistical validation -> RAG knowledge base rebuild.

Requires Ollama installed separately with the qwen2.5:0.5b model pulled
(see README) - this script does not start Ollama itself, since it's a
background service, not a one-shot job.

Usage:
    python scripts/run_pipeline.py
"""

import os
import subprocess
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
SCRIPTS_DIR = ROOT / "scripts"
DBT_DIR = ROOT / "dbt_project"
VALIDATION_DIR = ROOT / "validation"
RAG_DIR = ROOT / "rag"

DBT_ENV = os.environ.copy()
DBT_ENV["DBT_PROFILES_DIR"] = str(DBT_DIR)

# Resolve the dbt executable relative to the CURRENT python interpreter
# (sys.executable) rather than trusting PATH - on Windows, multiple
# Python installs on PATH can shadow each other (see README Section 9),
# so "dbt" alone is not reliable across machines/terminals.
_scripts_dir = Path(sys.executable).parent / "Scripts"
DBT_EXE = str(_scripts_dir / "dbt.exe") if (_scripts_dir / "dbt.exe").exists() else "dbt"

def run(label, cmd, cwd, env=None, allow_failure=False):
    print(f"\n{'=' * 70}\n{label}\n{'=' * 70}")
    result = subprocess.run(cmd, cwd=cwd, env=env or os.environ)
    if result.returncode != 0 and not allow_failure:
        print(f"\nPipeline FAILED at: {label}")
        sys.exit(1)

run("STEP 1/6: Generating synthetic raw data", [sys.executable, str(SCRIPTS_DIR / "generate_data.py")],
    SCRIPTS_DIR)
run("STEP 2/6: Loading raw data into DuckDB", [sys.executable, str(SCRIPTS_DIR /
    "load_raw_to_duckdb.py")], SCRIPTS_DIR)
run("STEP 3/6: Running dbt models (staging -> intermediate -> marts)",
    [DBT_EXE, "run"], DBT_DIR, DBT_ENV)
run("STEP 4/6: Running dbt tests (some FAILS/WARNs are expected - that IS the point of this project)",
    [DBT_EXE, "test"], DBT_DIR, DBT_ENV, allow_failure=True)
run("STEP 4b/6: Preserving test results for the RAG knowledge base",
    [sys.executable, "-c",
     "import shutil; shutil.copy('target/run_results.json', 'target/test_run_results.json')"],
    DBT_DIR)

```

```

run("STEP 5/6: Running Pandera statistical validation", [sys.executable, str(VALIDATION_DIR /
"pandera_checks.py")],
    VALIDATION_DIR, allow_failure=True)
run("STEP 6/6: Rebuilding the AI Copilot's RAG knowledge base", [sys.executable, str(RAG_DIR /
"build_knowledge_base.py")], RAG_DIR)

print(f"\n{'=' * 70}\nPipeline completed.")
print("Next steps:")
print(" 1. Make sure Ollama is running with the qwen2.5:0.5b model pulled.")
print(" 2. streamlit run dashboard/streamlit_app.py")
print(f"\n{'=' * 70}")

```

5.13 dashboard/streamlit_app.py

Why it exists. Two tabs over one DuckDB file: the quality dashboard and the copilot chat.

What the module says about itself. Two-tab app for the AI-Augmented Data Quality & Validation Framework:

Public functions in dashboard/streamlit_app.py

Function	What it does
load_warehouse_data()	—
load_dbt_results()	—
load_pandera_results()	—

Opening of the module

dashboard/streamlit_app.py

```

"""
streamlit_app.py
-----
Two-tab app for the AI-Augmented Data Quality & Validation Framework:

    Tab 1 - Data Quality Dashboard: dbt test results, Pandera statistical
            checks, the incident-day volume/flag-rate chart, and a
            filterable drill-through table over fct_orders.
    Tab 2 - AI Data Quality Copilot: a chat interface over the Local
            RAG pipeline (rag/copilot.py) - ask plain-English questions
            about test results, get grounded, cited answers.

Run with:
    streamlit run dashboard/streamlit_app.py
"""

import json
import sys
from pathlib import Path

import duckdb
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import streamlit as st

ROOT = Path(__file__).resolve().parent.parent
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from rag.copilot import ask # noqa: E402

DB_PATH = ROOT / "database" / "warehouse.duckdb"
DBT_TARGET = ROOT / "dbt_project" / "target"
REPORTS_DIR = ROOT / "reports"

st.set_page_config(page_title="AI Data Quality Platform", layout="wide", page_icon="📊")

@st.cache_data
def load_warehouse_data():
    conn = duckdb.connect(str(DB_PATH), read_only=True)
    orders = conn.execute("SELECT * FROM main.fct_orders").fetchdf()
    conn.close()
    orders["order_date"] = pd.to_datetime(orders["order_timestamp"]).dt.date
    return orders

@st.cache_data
def load_dbt_results():
    with open(DBT_TARGET / "test_run_results.json", "r", encoding="utf-8") as f:
        run_results = json.load(f)
    with open(DBT_TARGET / "manifest.json", "r", encoding="utf-8") as f:
        manifest = json.load(f)
    nodes = manifest.get("nodes", {})
    rows = []
    for r in run_results["results"]:

```

```

        name = nodes.get(r["unique_id"], {}).get("name") or r["unique_id"].split(".")[-1]
        rows.append({"test_name": name, "status": r["status"].upper(), "failures": r.get("failures", 0)
or 0})
    return pd.DataFrame(rows)

@st.cache_data
def load_pandera_results():
    with open(REPORTS_DIR / "pandera_validation_results.json", "r", encoding="utf-8") as f:
        return json.load(f)

orders = load_warehouse_data()
dbt_results = load_dbt_results()
pandera_results = load_pandera_results()

```

CHAPTER 6

The Incident Two Checks Could Not See

The central finding: why an aggregate rate threshold is structurally blind to a short, sharp incident, and why lowering the threshold is the wrong fix.

6.1 The incident

One day - 2026-06-17 - was deliberately corrupted. Order volume jumped from a 222-order daily average to 341. Twenty-two orders lost their customer reference and twenty-one referenced products that do not exist.

Every one of those 43 bad rows is on that single day. Nothing before or after it is affected.

6.1 Three checks, one incident, two different answers

Check	Granularity	Result
daily_volume_anomaly_detection	per day	FAIL - z = 4.56 on 2026-06-17
missing_customer_rate_threshold	whole dataset	PASS - 0.07% against a 2% threshold
orphan_product_rate_threshold	whole dataset	PASS - 0.06% against a 1% threshold

Two checks looked at the same corrupted data and pronounced it healthy. The difference between them and the check that caught it is not sophistication - it is the **denominator**.

6.2 The arithmetic

The missing-customer rate, measured as the check measures it:

$$\frac{22}{33542} = 0.00066 = 0.066\% < 2\%$$

The same 22 rows, measured against the day they occurred on:

$$\frac{22}{341} = 0.0645 = 6.45\% \gg 2\%$$

Nothing about the data changed. The incident is **98 times** more visible at daily granularity, and 98 is exactly $33542/341$ - the ratio of the denominators.

THE GENERAL FORM

Let r_d be the bad-row rate on the incident day, n_d the rows that day, and N the rows in the dataset. The aggregate rate is

$$r_{\text{agg}} = r_d \cdot \frac{n_d}{N}$$

The incident is attenuated by n_d/N - here about 1/98. **The aggregate check's sensitivity to a fixed-size incident decays as the dataset grows.** A check that works today stops working next year without anyone changing it.

6.3 Stronger: one of these checks can never fire

The project's report says an aggregate check would have missed this incident. The arithmetic supports something considerably stronger.

A threshold T over N rows fires only when bad rows exceed TN . A single-day incident can contribute at most n_d bad rows - the entire day. So the check is **capable** of detecting a one-day incident only if $n_d > TN$.

6.2 Detection capability of the two aggregate checks

Check	T	Rows needed	Rows available in one day	Verdict
Missing-customer	2%	670.8	341	can never fire
Orphaned-product	1%	335.4	341	needs $\geq$ 98.4% of the day

EVEN 100% CORRUPTION OF THAT DAY WOULD NOT TRIP THE 2% CHECK

341 of 33,542 rows is 1.02%. If every single order on 17 June had been missing its customer - total, catastrophic corruption of an entire day - the check would still have reported 1.02% against a 2% threshold and passed.

That check is not a badly-tuned control for single-day incidents. It is not a control for them at all, and no value you put in it above 0.66% would make it one.

6.4 Why lowering the threshold is the wrong fix

The instinct is to set $T = 0.5\%$. That does make the check fire on this incident. It also creates three new problems.

1. **It conflates two different failures.** A 0.5% threshold fires equally on one catastrophic day and on a chronic 0.5% background rate spread across the year. Those need different responses, and the

alert cannot tell you which you have.

2. **It expires.** The capability condition is $T < n_d/N$. As N grows the threshold must keep falling to preserve the same sensitivity. Nobody will remember to do that.
3. **It gets noisy, then muted.** As T approaches the background rate the check starts firing on ordinary variation, and an alert that fires on ordinary variation is an alert that will be switched off.

THE FIX IS THE DENOMINATOR, NOT THE NUMBER

Compute the rate *per day* and threshold that. The incident day reads 6.45% against any sane threshold and fires immediately. A chronic background problem reads its true rate every day. And the check's sensitivity no longer depends on how much history has accumulated - n_d does not grow, so a fixed threshold keeps working.

The general principle: a monitoring check must be scoped to the granularity at which the failure occurs. A daily incident needs a daily check. Aggregating over a period longer than the failure dilutes it by exactly the ratio of the periods, and that ratio is usually large.

This generalises well beyond data quality. It is why service monitoring alerts on error rate per minute rather than per month, and it is the same structure as Simpson's paradox - a relationship that exists in every subgroup can vanish when the subgroups are pooled. **Project 9** treats that case directly.

CHAPTER 7

Retrieval-Augmented Generation, From First Principles

Embeddings and what distance in vector space means, chunking, top-k retrieval, and why grounding a small model beats trusting a large one.

7.1 The problem RAG solves

A language model knows what was in its training data. It does not know that `assert_no_extreme_line_totals` returned 52 rows this morning, and it never will.

Two options. **Fine-tune** the model on your data - expensive, immediately stale, and it teaches the model to imitate your documents rather than to cite them. Or **retrieve** the relevant facts at question time and put them in the prompt.

RETRIEVAL WINS FOR THIS TASK ON FOUR COUNTS

It is **current** - rebuilding the knowledge base takes seconds after each pipeline run. It is **citable** - the retrieved chunks are shown to the user, so a claim can be checked. It is **correctable** - fixing a wrong answer means fixing a document, not retraining. And it works with a **tiny model**, because reading and summarising supplied text is a far easier task than recalling facts.

That last point is what makes a 0.5-billion-parameter model viable on a 6GB machine. The model is not the knowledge; it is the language interface to the knowledge.

7.2 Embeddings: meaning as geometry

An embedding maps text to a vector of a few hundred numbers such that **texts with similar meanings land near each other**. Not similar words - similar meanings. "The orphaned product test failed" and "orders referencing missing catalogue items" share almost no vocabulary and sit close together.

Similarity is then geometry. Two common measures:

$$\cos(u, v) = \frac{u \cdot v}{\|u\| \|v\|}$$

$$d(u, v) = \|u - v\|_2 = \sqrt{\sum_i (u_i - v_i)^2}$$

Cosine similarity is 1 for identical direction and 0 for orthogonal. L2 distance is 0 for identical vectors and grows with dissimilarity. Chroma reports a distance, so **smaller is better** - which is why the guardrail in the next chapter is an upper bound.

THE MODEL HERE IS MINILM, RUNNING LOCALLY VIA ONNX

Chroma's built-in embedding function is a MiniLM sentence transformer executing on the CPU. No API call, no key, no per-query cost, and it runs on the same modest hardware as everything else in this series.

7.3 Chunking, and what goes in the knowledge base

Embeddings work on passages, not documents. Too large a chunk and the vector averages several topics into a vague centroid that matches nothing well. Too small and it loses the context that made it meaningful.

Building the knowledge base

rag/build_knowledge_base.py

```

"""
build_knowledge_base.py
-----
Assembles the knowledge base the AI Data Quality Copilot retrieves from,
and embeds it into a persistent ChromaDB collection using Chroma's
built-in ONNX MiniLM embedding function (no torch/sentence-transformers
needed - a deliberate choice on RAM-constrained hardware, see README).

Four kinds of documents go in, each written to answer a different kind
of real question a teammate would actually ask:

1. dbt test results      -> "why did X fail / did X pass?"
2. Pandera check results -> "does anything look statistically off?"
3. Model documentation  -> "what does model Y even contain?"
4. A root-cause runbook  -> "what usually causes an orphaned product?"
5. A computed incident summary, built from REAL query results against
   the warehouse (not hand-written), so the copilot's most detailed
   answer is grounded in the actual data, not a canned story.

Run AFTER: scripts/generate_data.py, scripts/load_raw_to_duckdb.py,
dbt run, dbt test, validation/pandera_checks.py.
"""

import json
from pathlib import Path

import chromadb
import duckdb
import pandas as pd

ROOT = Path(__file__).resolve().parent.parent
DBT_TARGET = ROOT / "dbt_project" / "target"
REPORTS_DIR = ROOT / "reports"
DB_PATH = ROOT / "database" / "warehouse.duckdb"
CHROMA_PATH = ROOT / "rag" / "knowledge_base" / "chroma_store"
CHROMA_PATH.parent.mkdir(parents=True, exist_ok=True)

documents = [] # List of (id, text, metadata)

def add_doc(doc_id, text, **metadata):
    documents.append((doc_id, text.strip(), metadata))

# =====
# 1. dbt test results
# =====
DBT_TEST_EXPLANATIONS = {
    "assert_no_extreme_line_totals": (
        "Checks that every order's line_total falls between $0.50 and $5000, a plausible "
        "range for the NorthPeak Outdoor Gear catalog (products range roughly $15-$290). "
        "A failure here almost always indicates a PRICING BUG - most commonly a unit-price "
        "multiplier applied incorrectly upstream (e.g. a price stored in cents being read as "
        "dollars, or vice versa) - not a normal data-entry typo."
    ),

```

What gets embedded matters more than how it is chunked. The knowledge base is built from **the pipeline's own output**: dbt test results, the Pandera JSON, the incident summary, schema documentation and runbook entries.

THE KNOWLEDGE BASE IS REGENERATED FROM ARTEFACTS, NEVER HAND-WRITTEN

This is the single design decision that makes the copilot trustworthy. Because every chunk is derived from a file the pipeline produced, the copilot cannot be more out of date than the last run, and it cannot describe a test result that never happened.

A hand-written FAQ would drift from reality within a week, and the drift would be invisible - the copilot would keep answering confidently from stale text. This is the same principle as the book engine that produced this volume: generate the documentation from the thing it documents, so it cannot silently diverge.

7.4 Retrieval

Top-k retrieval

rag/copilot.py

```
def retrieve(question: str, top_k: int = TOP_K):
    collection = _get_collection()
    res = collection.query(query_texts=[question], n_results=top_k)
    hits = []
    for doc, dist, meta, doc_id in zip(
        res["documents"][0], res["distances"][0], res["metadatas"][0], res["ids"][0]
    ):
        hits.append({"id": doc_id, "text": doc, "distance": dist, "metadata": meta})
    return hits

def ask(question: str, model: str = OLLAMA_MODEL) -> dict:
```

The question is embedded with the same model as the documents - essential, since two embedding spaces are not comparable - and the four nearest chunks are returned with their distances.

`TOP_K = 4` is a trade-off. Too few and the answer misses context; too many and the prompt fills with marginally relevant text that a small model will happily quote from. With a 0.5B model, **context precision matters more than context recall**, because the model has little capacity to judge which parts of a long context are relevant.

CHAPTER 8

The Guardrail: Teaching a Model to Refuse

Why grounding alone does not prevent hallucination, how a distance threshold implements refusal, and what calibrating it honestly requires.

8.1 Retrieval always returns something

This is the flaw at the centre of naive RAG. A vector search for "what is the capital of France?" against a data-quality knowledge base returns four chunks - the four *least dissimilar* ones. It cannot return nothing.

Hand those to a model with an instruction to answer from context, and it will produce something. It has been given text and asked for an answer, so it will find a way - and the answer will arrive with citations attached.

A CITED HALLUCINATION IS WORSE THAN A PLAIN ONE

An ordinary wrong answer invites scepticism. An answer with sources beneath it invites trust, and the sources are real documents, so a user who spot-checks that they exist is reassured.

The citation UI increases trust; it does not increase accuracy. Whatever mechanism justifies that extra trust has to be built separately - which is what the guardrail is.

8.2 Refusal as a distance threshold

The guardrail constant and the system prompt

rag/copilot.py

```
DISTANCE_GUARDRAIL_THRESHOLD = 1.4 # calibrated empirically - see docs/architecture.md

SYSTEM_PROMPT = (
    "You are a Data Quality Copilot for the NorthPeak Outdoor Gear data warehouse. "
    "Answer the user's question using ONLY the CONTEXT provided below - do not use "
    "any outside knowledge, and do not make up numbers or test names that are not in "
    "the context. Be concise (2-4 sentences). If the context includes specific counts "
    "or percentages, cite them. If asked something the context doesn't cover, say so "
    "plainly instead of guessing."
)

_client = None
```

The rule is one line: if the *best* retrieved chunk is further away than 1.4, refuse without calling the model at all.

answer if $\min_i \text{mind}(q, c_i) \leq 1.4$, otherwise refuse

REFUSE BEFORE GENERATING, NOT AFTER

The check happens before the model is invoked. That matters for three reasons: it is deterministic, so the same question always gets the same decision; it is auditable, since the distance is a number you can log; and it cannot be argued out of, unlike a prompt instruction telling the model to decline.

Asking a model to police itself makes refusal a property of sampling. Checking a distance makes it a property of the system.

Note also the system prompt does real work alongside the guardrail: answer *only* from context, do not invent numbers or test names, cite counts where the context has them, say plainly when the context does not cover something. The guardrail handles the off-topic case; the prompt handles the partially-covered case.

8.3 What was measured, and what it is worth

Six test questions - in-scope and out-of-scope - all six handled correctly. The France question retrieved a best distance of 1.88, comfortably above 1.4, and was refused.

SIX IS A SMALL SAMPLE, AND THE PROJECT SAYS SO

6/6 is a real result and it is a weak one. Two calibration points either side of a threshold demonstrate that the threshold separates those two points. They say nothing about the queries in between - the tangentially related ones a real user actually asks, which are neither clearly on-topic nor obviously absurd.

A defensible calibration needs a labelled query set of perhaps fifty spanning the middle ground, with precision and recall reported at the chosen threshold, and ideally a sweep showing how they trade off. That is not in this project. **Project 11** builds it, and **Project 14** shows what happens without it - an LLM judge that approved 28 of 28 wrong answers.

8.4 Choosing a 0.5B model on purpose

`qwen2.5:0.5b` is small. A larger model would write better answers. The constraint driving the choice is stated in the source: this had to run on a 6GB-RAM machine, because a local-first copilot that only runs on a well-resourced dev box has missed its own point.

9.1 The trade-off, stated plainly

Property	0.5B local	Frontier API
Cost per query	zero	per token, forever
Data leaves the machine	never	yes
Runs offline	yes	no
Answer fluency	adequate, sometimes drifts	excellent
Reasoning over context	weak	strong

GROUNDING IS WHAT MAKES THE SMALL MODEL VIABLE

Asked to recall facts, a 0.5B model is unreliable. Asked to summarise four supplied paragraphs, it is adequate - and adequate is enough when the user can see the sources.

The architecture carries the correctness and the model carries the phrasing. That division is why the honest transcripts in [docs/](#) - including one where the model drifts - do not undermine the system: the drift is bounded to topics the knowledge base covers, and the citations let a reader catch it.

CHAPTER 9

AI-Assisted Rule Authoring, Including Where It Fails

The second GenAI use case: drafting validation rules from a table's profile, and the honest account of what a 0.5-billion-parameter model gets wrong.

9.1 The task

Writing validation rules for a new table is tedious and easy to skip. You must look at every column, decide what normal looks like, and express it as a check. Teams skip it, which is why so many tables have no tests at all.

`rag/suggest_validation_rules.py` profiles a raw table - column types, null rates, cardinality, ranges, sample values - hands that profile to the local model, and asks for candidate rules. The output is a **draft for a human to review**, written to `reports/ai_suggested_rules/`.

WHY GENERATION, NOT DETECTION

The model is not being asked to decide whether the data is good. It is being asked to propose what *could* be checked - a recall-oriented, low-stakes task where a wrong suggestion costs a reviewer ten seconds and a missed one costs nothing that was not already missing.

That framing is what makes a tiny model usable here. Given authority over the pipeline it would be dangerous; given a drafting role with a human gate it is genuinely useful.

9.2 Where the small model gets it wrong

The project reports this honestly rather than showing only the good suggestions. A 0.5-billion-parameter model produces three recognisable classes of error.

11.1 How the local model fails at rule drafting

Failure	What it looks like	Why a reviewer catches it
Plausible but wrong bound	a range that excludes legitimate values	the reviewer knows the business
Restating the profile	"check that null rate is 3%" - describing, not validating	it would pass forever by construction
Inventing a column	a rule for a field that does not exist	it fails immediately when run

THE THIRD FAILURE IS THE MOST INSTRUCTIVE

A rule referencing a nonexistent column is *self-detecting* - it errors the moment anyone runs it. The first is not. A subtly wrong bound looks like a real rule, passes review if the reviewer is skimming, and then fires on legitimate data forever until someone deletes it in irritation.

The dangerous AI output is the plausible one, which is why the human gate is a design requirement rather than a disclaimer. The same principle drives the citation requirement on the copilot: an answer you can check is safe in a way that a fluent answer is not.

CHAPTER 10

Worked Examples

The dilution arithmetic, the z-score, and the guardrail calibration, computed on this project's real numbers.

10.1 Example 1 - The rate that could not fire

All the figures here come from the project's own warehouse.

12.1 The incident, measured

Quantity	Value
Rows in <code>fct_orders</code>	33,542
Distinct days	151
Orders on 2026-06-17	341
Missing-customer rows, whole dataset	22
Missing-customer rows on the incident day	22 - all of them
Orphaned-product rows, whole dataset	21
Orphaned-product rows on the incident day	21 - all of them

The aggregate view

$$\text{rate}_{\text{missing}} = \frac{22}{33542} = 0.000656 = 0.066\% < 2\%$$

The daily view

$$\text{rate}_{\text{missing, 17 June}} = \frac{22}{341} = 0.0645 = 6.45\% \gg 2\%$$

THE SAME 22 ROWS ARE 0.066% OR 6.45% DEPENDING ONLY ON THE DENOMINATOR

Nothing about the data changed between those two lines. The incident is a factor of 98 more visible when the denominator is the day rather than the dataset - and 98 is just $33542/341$, the ratio of the two denominators.

10.2 Example 2 - Proving the check is incapable

Stronger than 'it missed this incident': for one of the two checks, no single-day incident could ever trip it.

A threshold T over N rows fires only when the number of bad rows exceeds TN . A one-day incident can contribute at most n_d bad rows - every order that day. So the check is **capable** of detecting a one-day incident only if

$$n_d > T \cdot N$$

12.2 Capability of each aggregate check

Check	Threshold T	Rows needed (TN)	Day has	Verdict
Missing-customer	2%	670.8	341	can never fire on a one-day incident
Orphaned-product	1%	335.4	341	fires only if >=98.4% of the day is bad

THE 2% CHECK IS NOT BADLY TUNED - IT IS STRUCTURALLY BLIND

Even if *every single order* on 17 June had a missing customer, 341 of 33,542 rows is 1.02%, still under the 2% threshold. No amount of severity on a single day can move that number past the line.

This is a much stronger statement than the project's report makes. The report says an aggregate check would have missed this incident. The arithmetic says the 2% check **cannot detect any single-day incident whatsoever** in a 151-day dataset - it is not a monitoring control at all for that failure class, whatever number you put in it.

The general condition

For a threshold to be able to catch a single bad day, you need $T < n_d/N \approx 1/151 = 0.66\%$. Any aggregate threshold above that is blind to single-day incidents by construction, and the blindness gets worse every day the dataset grows.

WHICH IS WHY THE FIX IS NOT A LOWER THRESHOLD

You could set $T = 0.5\%$. Then the check fires on a one-day incident - and also on a slow, chronic 0.5% background rate spread across the year, which is a different problem needing a different response. And in a year, when N has doubled, it is blind again.

The denominator is the bug, not the number. The fix is to compute the rate per day and threshold that - which converts a check whose sensitivity decays as the dataset grows into one whose sensitivity is constant.

10.3 Example 3 - Calibrating the guardrail threshold

The copilot refuses to answer when the best retrieved chunk's distance exceeds 1.4. That number is calibrated, and the calibration is two measurements.

12.3 The two calibration points

Query	Best distance	Relevant?	Behaviour at 1.4
"What happened on the incident day?"	well below 1.4	yes	answers - correct
"What is the capital of France?"	1.88	no	refuses - correct

The threshold sits between the two. Move it to 2.0 and the France question gets answered from data-quality documents - a hallucination with citations attached, which is worse than an ordinary one because it looks verified. Move it to 1.0 and legitimate questions phrased unusually start being refused.

TWO CALIBRATION POINTS IS NOT A CALIBRATION

A threshold separating one positive from one negative is a threshold that has been shown to work twice. It says nothing about the queries in between - the ones a real user actually asks, which are neither textbook-relevant nor obviously off-topic.

A defensible calibration needs a labelled set of perhaps fifty queries spanning relevant, tangential and irrelevant, and a reported precision and recall at the chosen threshold. The project tested 6 out of 6 correctly and that is what it claims - honest, and a small sample. **Project 11** builds the full evaluation harness this layer would need.

CHAPTER 11

Running the Project, Step by Step

Every terminal command needed to go from a fresh clone to a finished run, what each one does, and what to do when one of them fails.

11.1 Prerequisites

You need Python 3.11 or newer and Git. Verify both before starting - most reported 'the project does not work' turns out to be an environment that was never activated.

Check your toolchain

```
python --version
git --version
```

THIS PROJECT HAS ONE OPTIONAL LOCAL-LLM STAGE

Install Ollama and pull the model if you want the generative feature to run. It is optional: the project completes without it, and the deterministic control arm still produces results.

Optional - only for the LLM stage

```
ollama pull qwen2.5:0.5b
ollama list
```

11.2 Step 1 - Clone and enter the repository

Get the code

```
git clone https://github.com/Nikhil201716/05-AI-Augmented-Data-Quality-Validation-Framework.git
cd 05-AI-Augmented-Data-Quality-Validation-Framework
```

11.3 Step 2 - Create an isolated environment

A virtual environment keeps this project's dependencies from colliding with anything else on your machine.

Create and activate

```
python -m venv .venv

# Windows (PowerShell)
.venv\Scripts\Activate.ps1

# macOS / Linux
source .venv/bin/activate
```

IF POWERSHELL REFUSES TO RUN THE ACTIVATION SCRIPT

Windows blocks unsigned scripts by default. Allow them for your own user account only:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

11.4 Step 3 - Install dependencies

Install

```
pip install -r requirements.txt
```

11.5 Step 4 - Run the pipeline, stage by stage

Each command below produces the input the next one consumes. Run them in this order:

The full pipeline in order

```
python scripts/generate_data.py
python scripts/load_raw_to_duckdb.py
dbt run --project-dir dbt_project --profiles-dir dbt_project
dbt test --project-dir dbt_project --profiles-dir dbt_project
python validation/pandera_checks.py
python rag/build_knowledge_base.py
```

What each stage does:

1. `scripts/generate_data.py` - writes the seeded raw CSVs, including the injected 2026-06-17 incident
2. `scripts/load_raw_to_duckdb.py` - loads the raw CSVs into database/warehouse.duckdb as source tables
3. `dbt_project` - builds staging, intermediate and marts in dependency order
4. `dbt_project` - runs all 27 tests - expect PASS=24 WARN=1 ERROR=2

5. `validation/pandera_checks.py` - runs the five statistical checks and writes the JSON and markdown reports
6. `rag/build_knowledge_base.py` - embeds the test results and docs into the ChromaDB vector store

EXPECTED OUTPUT

Two dbt tests and two Pandera checks are SUPPOSED to fail - the dataset contains a deliberately injected incident, and a clean report would mean the validation layer is broken. Expect PASS=24 WARN=1 ERROR=2 from dbt and 3/5 from Pandera, with the volume anomaly naming 2026-06-17 at z=4.56. The copilot needs Ollama running and qwen2.5:0.5b pulled; without it every other stage still completes.

11.6 Run everything in one command

Executes every stage above in order. Use this for a clean rebuild.

```
python scripts/run_pipeline.py
```

11.7 Ask the copilot from the command line

Prints the answer with its retrieved sources and their distances. Try an off-topic question to see the guardrail refuse instead of guessing.

```
python rag/copilot.py "What happened on the incident day?"
```

11.8 Draft validation rules with the local model

Profiles a raw table and writes candidate rules to `reports/ai_suggested_rules/` for human review. Read them critically - Chapter 11 covers the three ways this model gets it wrong.

```
python rag/suggest_validation_rules.py
```

11.9 Launch the dashboard and chat UI

Opens on `http://localhost:8501` with a Data Quality tab and an AI Copilot tab, both reading the same DuckDB file.

```
streamlit run dashboard/streamlit_app.py
```

11.10 Verify reproducibility from nothing

A clean rebuild should reproduce 33,542 orders, the same 24/1/2 dbt outcome, and the same z=4.56 on 2026-06-17.

```
rm -rf data database reports rag/knowledge_base
python scripts/run_pipeline.py
```

11.11 Command reference card

Every command from this chapter in one block, for when you already know what you want.

Complete command reference

```
# --- setup (once) ---
git clone https://github.com/Nikhil201716/05-AI-Augmented-Data-Quality-Validation-Framework.git
cd 05-AI-Augmented-Data-Quality-Validation-Framework
python -m venv .venv
.venv\Scripts\Activate.ps1          # Windows
source .venv/bin/activate          # macOS / Linux
pip install -r requirements.txt

# --- run ---
python scripts/generate_data.py
python scripts/load_raw_to_duckdb.py
dbt run --project-dir dbt_project --profiles-dir dbt_project
dbt test --project-dir dbt_project --profiles-dir dbt_project
python validation/pandera_checks.py
python rag/build_knowledge_base.py

# --- extras ---
python scripts/run_pipeline.py
python rag/copilot.py "What happened on the incident day?"
python rag/suggest_validation_rules.py
streamlit run dashboard/streamlit_app.py
rm -rf data database reports rag/knowledge_base
python scripts/run_pipeline.py

# --- finish ---
deactivate
```

11.12 Troubleshooting

Common problems and their fixes

Symptom	Cause	Fix
<code>ModuleNotFoundError</code>	environment not activated	re-run the activate command from Step 2
<code>FileNotFoundError</code> on a data file	an earlier stage was skipped	run the stages in the documented order
Numbers differ from this book	a seed or parameter was changed	check the constants at the top of the generator
<code>command not found</code>	dependency missing from this env	<code>pip install -r requirements.txt</code> with the env active
Permission denied writing output	directory created by another user or process	delete the generated output directory and re-run

CHAPTER 12

The Dashboard and the Transcripts

Two audiences again, and why unedited transcripts are the load-bearing evidence for an AI feature.

12.1 One warehouse, two tabs

`dashboard/streamlit_app.py` reads the same DuckDB file dbt builds into and offers two views: a Data Quality Dashboard showing test results and the incident, and an AI Copilot Chat.

The most useful panel is the incident chart - daily order volume against daily data-quality flag rate. Both spike on 17 June and only there. It makes the argument of Chapter 6 visible: the incident is obvious at daily granularity and invisible in aggregate.

Launch the dashboard and copilot

```
streamlit run dashboard/streamlit_app.py
```

12.2 Why the transcripts matter more than the dashboard

Any claim about an LLM feature is unverifiable unless the actual input and output are shown. `docs/copilot_transcript_examples.md` contains real, unedited exchanges - including one where the model drifts slightly off-topic.

PUBLISHING THE IMPERFECT TRANSCRIPT IS THE CREDIBLE MOVE

A page of flawless answers from a 0.5-billion-parameter model would invite exactly the right suspicion: which attempts were not shown? Including a drift case answers that question before it is asked, and it is also the more useful document - it shows a reader what this model class actually does rather than what it can be made to do on a good run.

It also makes the guardrail's job legible. The refusal on "what is the capital of France" is not a party trick; it is the mechanism that keeps the drift bounded to topics the knowledge base actually covers.

CHAPTER 13

Results, and What They Do Not Establish

What was proved, and the limits of each claim.

13.1 What was proved

1. **A layered dbt pipeline builds a warehouse and tests it.** 27 tests, 33,542 orders, real results with two genuine failures.
2. **Two validation tools catch disjoint failure classes.** dbt found 21 orphaned foreign keys at the row level; Pandera found a distributional anomaly no row-level test could express.
3. **An aggregate rate threshold is structurally blind to a short incident.** Proved, not merely observed: the 2% check could not have fired even at 100% corruption of the affected day.
4. **A local RAG copilot answers from real test output and refuses off-topic questions.** 6/6 on the tested set, with a pre-generation distance guardrail.
5. **The whole pipeline is reproducible.** A clean rebuild after deleting the data, database, reports and vector store produced identical row counts, identical test outcomes and the same $z=4.56$.

13.2 What this does not establish

1. **That the guardrail threshold is well calibrated.** Two points either side of 1.4 and six correct answers. No labelled set, no precision/recall, no sweep.
2. **That the copilot is accurate in general.** It was tested on questions whose answers are in the knowledge base. Its behaviour on partially-covered questions - the common real case - is not measured.
3. **That the plausibility bounds are right.** They were chosen to match the generator's parameters, so their success is circular.
4. **That the z-score check would catch a subtler incident.** The baseline includes the outlier, which inflates the standard deviation and biases the check against itself. It fired because the incident was large.
5. **That the marts contain only good data.** They demonstrably do not - `dbt run` precedes `dbt test`, so the 21 orphaned rows and 52 extreme totals are in `fct_orders`.
6. **That any of this holds at scale.** 33,542 rows in an in-process database on a laptop.

TWO SUBSTITUTIONS, BOTH DISCLOSED IN THE PROJECT ITSELF

DuckDB for a warehouse, because everything here must run for free on modest hardware.

Pandera for Great Expectations, because GX's available release pins numpy<2.0, which cannot build on Python 3.14 here without a C compiler.

The second is the more interesting disclosure: the original tool choice was the industry-standard one, it failed for a specific and checkable environmental reason, and the substitute covers the same ground. Saying that is more useful to a reader than silently presenting Pandera as the plan all along.

CHAPTER 14

Exercises, with Solutions

Problems requiring the chapters, with worked answers.

14.1 Exercises

1. The z-score check computes its baseline over all days including the incident. Name two fixes and say which you would ship.
2. Prove that the 2% missing-customer check cannot fire on any single-day incident in this dataset. Then find the largest threshold that could.
3. The dataset doubles to 300 days at the same daily volume. What happens to the aggregate check's ability to detect a one-day incident?
4. Why is `not_null` on `customer_id` a warning while the product relationship test is an error? Give the business reason for each.
5. A colleague proposes deleting the Pandera layer since dbt already tests the data. Give a concrete failure that only Pandera catches, and one that only dbt catches.
6. The guardrail threshold is raised from 1.4 to 2.0. What happens to the France question, and why is that outcome worse than a plain hallucination?
7. Why must the question be embedded with the same model as the documents?
8. Design the calibration this project is missing: what set would you build, what would you measure, and what would you report?

14.2 Solutions

1. The contaminated baseline

Fix A: robust statistics - use the median and the median absolute deviation, which a single outlier barely moves. Fix B: leave-one-out - compute each day's baseline from the other 150 days, so a day can never inflate the yardstick used to judge it. **Ship A.** It is a two-line change, needs no extra passes over the data, and is standard practice for outlier detection. B is more precise and costs a recomputation per day for a benefit that only matters when several outliers cluster.

2. The impossibility proof

The check fires when bad rows exceed $TN = 0.02 \times 33542 = 670.8$. A single day contains 341 orders, so it can contribute at most 341 bad rows. Since $341 < 670.8$, no single-day incident can trip it - even total corruption gives 1.02%. The largest workable threshold is $T < n_d/N = 341/33542 = 1.02\%$, and to fire on a merely severe day rather than a totally corrupt one you need far less - a 50%-corrupted day needs $T < 0.51\%$.

3. Doubling the dataset

It gets strictly worse. n_d is unchanged but N doubles to ~67,000, so the attenuation factor n_d/N halves and the threshold required for capability halves with it. **The check silently loses sensitivity as history accumulates**, with no code change and no warning - which is the strongest practical argument for scoping the check per day, where n_d is the denominator and does not grow.

4. The two severities

Guest and anonymous checkout is a normal part of an e-commerce business, so a background rate of null `customer_id` is expected; failing the pipeline on it would fail most days and the test would be disabled within a month. An order referencing a product that is not in the catalogue has no legitimate explanation - it means a broken join, a deleted product, or corrupt input - so it is an error. Severity should encode *whether a legitimate business process produces this*, not how unusual it looks.

5. The disjoint failures

Only Pandera: a unit price of \$4,999 on a \$49.99 item. Not null, unique, positive, resolves to a real product - every structural test passes, and the value is nonsense. Only dbt: one order whose `product_id` does not exist. One row in 33,542 moves no distribution, but the relationship test catches it exactly. The tools are not redundant because their blind spots do not overlap.

6. Raising the guardrail

1.88 now falls under the threshold, so the France question is answered - from data-quality documents, with those documents cited beneath the answer. It is worse than a plain hallucination because the citation UI signals verification: a user who checks that the sources exist finds that they do, and is reassured by evidence that has nothing to do with whether the answer is right. **Presenting unfounded output inside a trust-signalling interface converts a visible failure into an invisible one.**

7. One embedding space

A distance is only meaningful between vectors in the same space. Two models produce different geometries - even the same dimensionality carries no shared meaning across models - so distances between them are arbitrary numbers. The retrieval would return essentially random chunks, and, worse, it would not error: you would get plausible output grounded in irrelevant context.

8. The missing calibration

Build ~50 labelled questions in three bands: clearly answerable from the knowledge base, tangential (about the warehouse but not covered), and clearly out of scope. For each, record the best retrieval distance. Sweep the threshold and report precision and recall of the refusal decision at each value, then choose the operating point deliberately - for this application, favouring refusal, since a wrong answer about data quality is more costly than an unnecessary 'I don't know'. Report the curve, not just the chosen point, so the choice is visible.

CHAPTER 15

Appendix

Repository map, glossary, and reproducibility notes.

15.1 A - Repository map

Where everything lives

Path	Purpose
scripts/generate_data.py	seeded generation with the injected incident
scripts/load_raw_to_duckdb.py	raw CSV to DuckDB source tables
scripts/run_pipeline.py	one-command orchestration of every stage
dbt_project/models/staging/	one model per source: rename and cast only
dbt_project/models/intermediate/	joins and reusable business logic
dbt_project/models/marts/	fct_orders, dim_customers, dim_products
dbt_project/models/*/_*.yml	generic tests and documentation
dbt_project/tests/	two singular tests written as SQL
validation/pandera_checks.py	the five statistical checks
rag/build_knowledge_base.py	embeds pipeline output into ChromaDB
rag/copilot.py	retrieval, guardrail, grounded generation
rag/suggest_validation_rules.py	AI-assisted rule drafting
dashboard/streamlit_app.py	dashboard and copilot chat
reports/data_quality_assessment.md	the business-facing findings report
reports/pandera_validation_results.json	structured results, also the RAG source
docs/copilot_transcript_examples.md	real, unedited copilot transcripts
docs/architecture.md	design rationale and disclosed trade-offs

15.2 B - Glossary

Terms used in this book

Term	Meaning
Seeded generator	a synthetic data producer whose random draws are reproducible from a fixed starting value
Ground truth	the mechanism deliberately injected into synthetic data, used only to score an analysis - never to make a decision
Control arm / baseline	a simple alternative a new method must beat before it is worth its complexity
Point-in-time correctness	computing a feature using only information that existed at the moment being modelled
Leakage	information from the future or from the target entering a feature, inflating every offline metric
Reproducibility checksum	a hash of generated output, compared across a fresh process to prove determinism
dbt	a tool that turns SELECT statements into a managed, tested, dependency-ordered warehouse build
Model	one SELECT statement in a file, materialised as a view or table
ref()	dbt's cross-model reference, from which the dependency graph is derived
Materialisation	whether a model becomes a view (recomputed) or a table (stored)
Staging / intermediate / marts	the standard layering: tidy sources, join logic, consumption tables
Generic test	a reusable test declared in YAML against a column
Singular test	a hand-written SQL test selecting rows that violate a rule
Severity	whether a failing test warns or errors - a business decision, not a default
DuckDB	an in-process columnar analytical database; no server required
Pandera	a statistical dataframe validation library, used here in place of Great Expectations
Structural vs statistical validation	row-level constraints versus distributional plausibility
Denominator dilution	an incident's rate shrinking as the aggregation window grows
z-score	how many standard deviations an observation sits from the mean
MAD	median absolute deviation - a robust alternative to the standard deviation
RAG	retrieval-augmented generation: fetch relevant text, then generate from it
Embedding	a vector representation of text where nearby means similar in meaning

Term	Meaning
Vector store	a database indexed for nearest-neighbour search over embeddings
Chunking	splitting documents into passages small enough to embed meaningfully
Top-k retrieval	returning the k nearest chunks to a query
Grounding	requiring a model to answer only from supplied context
Hallucination guardrail	a mechanism that refuses when nothing relevant was retrieved
Cited hallucination	an unfounded answer presented with real sources attached

15.3 C - Reproducibility

Every dataset in this project is produced by a seeded generator, so the whole pipeline can be rebuilt from nothing and should produce byte-identical output.

A FIXED SEED IS A CLAIM, NOT A GUARANTEE

Two projects in this portfolio shipped generators with fixed seeds that were still not reproducible across runs. The causes were `rng.choice(list(a_set))` and `abs(hash(some_string))` - both of which are perfectly stable *within* one process and vary between processes, because Python randomises string hashing per interpreter run.

Neither is visible from reading the code, from the seed, or from the output looking sensible. Both were caught only by hashing the generated files and comparing across a *fresh* process. The defences used throughout this portfolio are to sort any collection before it feeds a random draw, and to verify by checksum in a new process rather than trusting the seed.

15.4 D - Where this leads next

This project pairs analytics engineering with retrieval. The thread continues: **Project 8** extends dbt into a semantic layer with data contracts and lineage. **Project 9** treats the aggregation trap of Chapter 6 in its general form as Simpson's paradox, and builds the experimentation statistics this book only gestured at. **Project 11** builds the RAG evaluation harness this copilot needs - the labelled set, the precision/recall sweep, the calibrated operating point. **Project 12** compares vector retrieval against a knowledge graph and finds a query class where vector search scores 0.000. **Project 14** shows the cost of skipping evaluation entirely: an LLM judge that approved 28 of 28 wrong answers. **Project 15** governs the metric definitions themselves.

The three most valuable improvements to *this* project, in order: make the aggregate rate checks per-day, which converts two structurally blind checks into working ones; switch the volume z-score to median and MAD so the outlier stops inflating its own yardstick; and build the labelled query set the guardrail threshold is currently missing.